

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-a570d8e0028ea7bee.jsonl`
- SHA-256: `d6edeaf021cbeeb4a63f456e9744c35eb07d322463220d59589e820f849809af`
- Source modified: `2026-06-22T01:31:48+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_2be29ecf-a31`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:30:51.582Z)

Adversarially verify this P2b-install review finding. Read the cited code in C:\Users\avidu\Projects\bhi-p2b yourself (run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and open the file). Decide if it is a REAL defect that should block or be fixed, or a false alarm / nit. Default to is_real=false if it does not hold up.

Lens: plan-consistency-regression

Severity(claimed): nit

Title: Duplicated default_os_app_home has no parity test guarding drift from the engine resolver

File: C:\Users\avidu\Projects\bhi-p2b\scripts\install.py

Detail: scripts/install.py and scripts/uninstall.py each re-implement default_os_app_home() and hardcode APP_DIR_NAME='Khelsutra' as a standalone mirror of

backend.utils.app_home.default_os_app_home (the stdlib-only-bootstrap justification is sound). I verified all three implementations agree TODAY (same per-OS bases, same APPDATA/XDG precedence, same 'Khelsutra' dir name). The risk is purely future drift: if backend.utils.app_home ever changes the app-dir name or precedence, the launcher would pin a RALLY_APP_HOME that diverges from where the engine's own resolver looks, silently splitting state. There is no test asserting parity between the script mirror and the engine source of truth.

Suggested fix: Optional: add a test that imports backend.utils.app_home and asserts

install_mod.default_os_app_home(...) (for each of Windows/Darwin/Linux with fixed env) equals the engine's resolver output, and that APP_DIR_NAME matches. Low priority ? current values are identical and verified.

2. assistant (2026-06-22T01:30:53.886Z)

I'll verify this finding by examining the actual code in the cited repository.

3. user (2026-06-22T01:30:57.160Z)

```
README.md | 31 ++++++
backend/config/__init__.py | 9 +-
backend/config/loader.py | 34 ++++++
backend/main.py | 15 +++
docs/PACKAGING_PLAN.md | 13 +-
scripts/install.py | 210 ++++++
scripts/uninstall.py | 150 ++++++
tests/test_app_launcher.py | 42 ++++++
tests/test_p2b_install.py | 163 ++++++
9 files changed, 660 insertions(+), 7 deletions(-)
```

4. assistant (2026-06-22T01:30:59.852Z)

Let me read the install.py and uninstall.py implementations, plus the engine source of truth.

5. user (2026-06-22T01:31:00.349Z)

```
1      #!/usr/bin/env python3
2      """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
3      P2b).
4      Cross-platform (Windows / macOS / Linux), no native component (01):
5
6      * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7      env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8      * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9      ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10     (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11     starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12     launcher (not by the engine) roots the ``.env`` in the app-home;
13     * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15     The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16     ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17     this script only wires the install + the launcher.
18
19     Usage::
20
21         python scripts/install.py                # install '.' (this checkout), uv-
if-present else pip
22         python scripts/install.py --target badminton-highlight-indexer  # from PyPI (once
published)
23         python scripts/install.py --method pip      # force pip
24         python scripts/install.py --dry-run         # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)
-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
exists)."""
51         system = system or platform.system()
52         e = os.environ if env is None else env
53         if system == "Windows":
```

```

54         base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
55     elif system == "Darwin":
56         base = str(Path.home() / "Library" / "Application Support")
57     else:
58         base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
59     return Path(base) / app_name
60
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
tuple[str, str, bool]:
63         """Return `(filename, content, make_executable)` for the per-OS launcher shortcut.
64
65         The shortcut pins `RALLY_APP_HOME` then runs `indexer --app`. PURE (no IO) so
it's testable.
66         """
67         home = str(app_home)
68         if system == "Windows":
69             content = (
70                 "@echo off\r\n"
71                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
the app.\r\n"
72                 f'set "RALLY_APP_HOME={home}"\r\n'
73                 f'{indexer_cmd} --app\r\n"
74             )
75             return "KhelSutra.bat", content, False
76         if system == "Darwin":
77             content = (
78                 "#!/bin/bash\n"
79                 "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
80                 f'export RALLY_APP_HOME="{home}"\n'
81                 f"exec {indexer_cmd} --app\n"
82             )
83             return "KhelSutra.command", content, True
84         # Linux / other
85         content = (
86             "#!/bin/bash\n"
87             "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
app.\n"
88             f'export RALLY_APP_HOME="{home}"\n'
89             f"exec {indexer_cmd} --app\n"
90         )
91         return "KhelSutra.sh", content, True
92
93
94     def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
95                       timestamp: str) -> dict:
96         """The uninstall manifest (PURE). `created` = files THIS installer made (safe to
remove);
97         `user_data` = paths the engine/user own (kept unless `uninstall.py --purge`)."""
98         return {
99             "schema": 1,
100             "product": "KhelSutra highlight indexer",
101             "package": PACKAGE,
102             "install_method": method,          # "uv" | "pip"
103             "indexer_command": "indexer",
104             "app_home": str(app_home),
105             "shortcut": str(shortcut),
106             "created": [str(p) for p in created],
107             "user_data": [
108                 str(app_home / "config.json"),
109                 str(app_home / ".env"),
110                 str(app_home / "output"),
111             ],
112             "timestamp": timestamp,

```

```

113     }
114
115
116     def resolve_method(requested: str, uv_present: bool) -> str:
117         """Pick the installer: explicit beats auto; auto prefers uv when present, else
118         pip."""
119         if requested in ("uv", "pip"):
120             return requested
121         return "uv" if uv_present else "pip"
122
123     def install_command(method: str, target: str) -> list[str]:
124         """The argv for the chosen installer (PURE)."""
125         if method == "uv":
126             return ["uv", "tool", "install", target]
127         return [sys.executable, "-m", "pip", "install", target]
128
129
130     # --- IO / orchestration
131     -----
132
133     def _run(argv: list[str]) -> None:
134         print(f" $ {' '.join(argv)}")
135         subprocess.run(argv, check=True)
136
137     def main(argv: list[str] | None = None) -> int:
138         ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer
139         (LIGHT tier).")
140         ap.add_argument("--target", default=".",
141             help="What to install: '.' (this checkout, default) or a PyPI
142             name/spec.")
143         ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
144             help="Installer to use (default: uv if present, else pip).")
145         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
146             nothing.")
147         args = ap.parse_args(argv)
148
149         system = platform.system()
150         app_home = default_os_app_home()
151         method = resolve_method(args.method, shutil.which("uv") is not None)
152         cmd = install_command(method, args.target)
153         fname, content, executable = shortcut_spec(system, app_home)
154         shortcut = app_home / fname
155
156         print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
157         print(f" OS          : {system}")
158         print(f" installer   : {method}")
159         print(f" target      : {args.target}")
160         print(f" app-home    : {app_home}")
161         print(f" shortcut    : {shortcut}")
162         if args.dry_run:
163             print("\n[dry-run] would run:")
164             print(f" $ {' '.join(cmd)}")
165             print(f" write shortcut -> {shortcut}")
166             print(f" write manifest -> {app_home / MANIFEST_NAME}")
167             print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
168             return 0
169
170         print("\n[1/3] installing the package?")
171         try:
172             _run(cmd)
173         except (subprocess.CalledProcessError, FileNotFoundError) as e:
174             print(f"[ERROR] install failed: {e}")
175             if method == "uv":

```

```

173         print(" (uv not usable? re-run with --method pip)")
174     return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KhelSutra:")
200     if system == "Windows":
201         print(f"    double-click {shortcut}    (or run `indexer --app`)")
202     else:
203         print(f"    {shortcut}    (or run `indexer --app`)")
204     print("    The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f"    To uninstall: python \"{dst_uninstall}\"    (add --purge to also delete
your data)")
206     return 0
207
208
209 if __name__ == "__main__":
210     raise SystemExit(main())
211

```

6. user (2026-06-22T01:31:00.889Z)

```

1     #!/usr/bin/env python3
2     """Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).
3
4     Reverses ``scripts/install.py`` from the manifest it wrote:
5
6     * uninstalls the package (``uv tool uninstall`` / ``pip uninstall -y`` ? matches the
install method);
7     * removes the files the installer created (launcher shortcut, the copied uninstaller,
the manifest);
8     * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the
Gemini key,
9     and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.
10
11     Usage::
12
13     python uninstall.py                # find the manifest in the app-home, keep user
data
14     python uninstall.py --purge        # also delete config.json / .env / output (your
data)
15     python uninstall.py --manifest /path/to/uninstall_manifest.json
16     python uninstall.py --dry-run      # print the plan, change nothing

```

```

17
18     stdlib-only; safe to run from the copy the installer dropped in the app-home.
19     """
20     from __future__ import annotations
21
22     import argparse
23     import json
24     import os
25     import platform
26     import shutil
27     import subprocess
28     import sys
29     from pathlib import Path
30
31     APP_DIR_NAME = "Khelsutra"
32     MANIFEST_NAME = "uninstall_manifest.json"
33
34
35     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
36                             env: dict[str, str] | None = None) -> Path:
37         """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not
38         given)."""
39         system = system or platform.system()
40         e = os.environ if env is None else env
41         if system == "Windows":
42             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
43         elif system == "Darwin":
44             base = str(Path.home() / "Library" / "Application Support")
45         else:
46             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
47         return Path(base) / app_name
48
49     def find_manifest(explicit: str | None) -> Path | None:
50         """The manifest path: explicit arg, else next to THIS file, else the default app-
51         home."""
52         if explicit:
53             p = Path(explicit)
54             return p if p.exists() else None
55         for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
56                     default_os_app_home() / MANIFEST_NAME):
57             if cand.exists():
58                 return cand
59         return None
60
61     def uninstall_command(method: str, package: str) -> list[str]:
62         """The argv to remove the package (PURE)."""
63         if method == "uv":
64             return ["uv", "tool", "uninstall", package]
65         return [sys.executable, "-m", "pip", "uninstall", "-y", package]
66
67
68     def _rm(path: Path, dry: bool) -> None:
69         try:
70             if path.is_dir() and not path.is_symlink():
71                 print(f" - rmtree {path}")
72                 if not dry:
73                     shutil.rmtree(path, ignore_errors=True)
74             elif path.exists() or path.is_symlink():
75                 print(f" - remove {path}")
76                 if not dry:
77                     path.unlink(missing_ok=True)
78         except OSError as e:
79             print(f" ! could not remove {path}: {e}")

```

```

80
81
82     def main(argv: list[str] | None = None) -> int:
83         ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight
indexer.")
84         ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
85         ap.add_argument("--purge", action="store_true",
86                         help="Also delete YOUR data (config.json, .env, output/). Default
keeps it.")
87         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
nothing.")
88         args = ap.parse_args(argv)
89
90         manifest_path = find_manifest(args.manifest)
91         if not manifest_path:
92             print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
93             return 1
94         manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
95         method = manifest.get("install_method", "pip")
96         package = manifest.get("package", "badminton-highlight-indexer")
97         created = [Path(p) for p in manifest.get("created", [])]
98         user_data = [Path(p) for p in manifest.get("user_data", [])]
99         app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
100
101         print(f"KhelSutra uninstaller ? {package} (installed via {method})")
102         print(f"  manifest : {manifest_path}")
103         print(f"  app-home : {app_home}")
104         print(f"  data      : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
--purge to delete)'}")
105         dry = args.dry_run
106
107         print("\n[1/3] removing the package?")
108         cmd = uninstall_command(method, package)
109         print(f"  $ {' '.join(cmd)}")
110         if not dry:
111             try:
112                 subprocess.run(cmd, check=False)
113             except FileNotFoundError as e:
114                 print(f"  ! {method} not found ({e}); remove the package manually.")
115
116         print("\n[2/3] removing installer-created files?")
117         # Remove the manifest LAST (we still reference it); skip the running uninstaller
copy itself.
118         self_path = Path(__file__).resolve()
119         for p in created:
120             if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
121                 continue
122             _rm(p, dry)
123
124         if args.purge:
125             print("\n[3/3] purging user data (--purge)?")
126             for p in user_data:
127                 _rm(p, dry)
128         else:
129             print("\n[3/3] keeping your data:")
130             for p in user_data:
131                 print(f"  · {p}")
132
133         # Manifest + (optionally) the empty app-home last.
134         _rm(manifest_path, dry)
135         if args.purge and app_home.exists():
136             try:
137                 remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
138                 if not remaining:
139                     _rm(app_home, dry)

```

```

140         except OSError:
141             pass
142
143         print("\n? Uninstalled." + (" " if args.purge else " Your data was kept. "))
144         if self_path.parent == app_home:
145             print(f"    (you can delete this script + the app-home folder manually:
{app_home})")
146         return 0
147
148
149     if __name__ == "__main__":
150         raise SystemExit(main())
151

```

7. assistant (2026-06-22T01:31:03.822Z)

Now let me find and read the engine source of truth, `backend/utils/app\_home.py`.

8. user (2026-06-22T01:31:05.303Z)

""""Resolved application-data HOME for a packaged, batteries-included install.

PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``, the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it from the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user silently "loses" their index. This module introduces ONE resolver every state path roots on.

Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:

- * ``RALLY\_APP\_HOME`` env set ? that directory is the app home (the packaged launcher sets it).

- * ``RALLY\_APP\_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a behaviour.

The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` / ``~/.local/share``) is computed by :func:`default\_os\_app\_home` but is **NOT** auto-selected here ? the launcher (P2) calls it and exports ``RALLY\_APP\_HOME`` explicitly. Keeping the resolver opt-in means importing this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise writes into a real user profile during a test run).

stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ? before the heavier pydantic-backed ``backend.config`` package ? keeping the early ``load\_dotenv`` path light.

""""

```
from __future__ import annotations
```

```
import os
```

```
import platform
```

```
from pathlib import Path
```

#: Environment variable the packaged launcher sets to pin the app-data home.

```
APP_HOME_ENV = "RALLY_APP_HOME"
```

#: OS-app-dir leaf used by :func:`default\_os\_app\_home` (the product brand).

```
APP_DIR_NAME = "Khelsutra"
```

```
def app_home() -> Path:
```

```
    """The resolved application-data home.
```

```
    ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD, byte-identical
```

```
    to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
```

```
    """
```

```
    env = os.environ.get(APP_HOME_ENV)
```

```

if env and env.strip():
    return Path(env.strip()).expanduser()
return Path(".")

def app_path(*parts: str | os.PathLike[str]) -> Path:
    """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
    return app_home().joinpath(*[str(p) for p in parts])

def default_config_path() -> Path:
    """``<app-home>/config.json`` ? the config file the loader reads and ``--setup`` writes.

    With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative), unchanged.
    """
    return app_home() / "config.json"

def default_env_path() -> Path:
    """``<app-home>/env`` ? the dotenv file the server prefers (e.g. ``GEMINI_API_KEY``)."""
    return app_home() / ".env"

def env_file_to_load() -> Path | None:
    """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's DEFAULT
    (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()`` behaviour.

    Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file exists. With
    the
    env UNSET this returns ``None`` so the caller keeps the historical search and does NOT newly
    prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file, not the
    CWD).
    """
    home = app_home()
    if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv() search
        return None
    candidate = home / ".env"
    return candidate if candidate.exists() else None

def resolve_output_dir(output_dir: str) -> str:
    """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one through.

    With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default ``"output"`` when
    empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no pathlib
    normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir like
    ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the rest of
    the
    app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` / ``C:x``) ?
    passes
    through untouched.
    """
    home = app_home()
    if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
        return output_dir or "output"
    if not output_dir:
        return str(home / "output")
    if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
        return output_dir
    return str(home / output_dir)

def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
    """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
    ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`)."

```

```

* Windows ? ``%APPDATA%\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
* macOS   ? ``~/Library/Application Support/<app_name>``
* Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if unset)
"""
system = platform.system()
if system == "Windows":
    base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
elif system == "Darwin":
    base = str(Path.home() / "Library" / "Application Support")
else:
    base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
return Path(base) / app_name
---FILES---
__init__.py
__pycache__
app_home.py
ffmpeg.py
geometry.py
hashing.py
logging_setup.py
paths.py
redact.py
run_telemetry.py
single_instance.py
video.py
workspace.py

```

9. assistant (2026-06-22T01:31:14.222Z)

I now have all three implementations. Let me verify the parity claim precisely by comparing them side by side.

Engine `backend/utils/app\_home.py::default\_os\_app\_home`:

```

- `APP_DIR_NAME = "Khelsutra"`
- Windows: `os.environ.get("APPDATA") or str(Path.home()/"AppData"/"Roaming")` ? `~/ app_name`
- Darwin: `Path.home()/"Library"/"Application Support"` ? `~/ app_name`
- else: `os.environ.get("XDG_DATA_HOME") or str(Path.home()/"local"/"share")` ? `~/ app_name`
- Reads `platform.system()` and `os.environ` directly (no injectable params).

```

Installer `scripts/install.py::default\_os\_app\_home`:

```

- `APP_DIR_NAME = "Khelsutra"`
- Same per-OS bases, same precedence, same leaf ? but adds injectable `system` and `env` params (a superset; defaults reproduce engine behavior).

```

Uninstaller `scripts/uninstall.py::default\_os\_app\_home`: identical to the installer's.

The finding's factual claims hold: three mirrors agree today, the bootstrap justification (stdlib-only, runs before the package exists) is genuinely sound, and there is no parity test. Let me confirm the no-test claim.

10. user (2026-06-22T01:31:22.126Z)

=== app_home refs in tests ===

```

tests/test_app_home.py:12:from backend.utils import app_home as ah
tests/test_app_home.py:17:def _clear_app_home_env(monkeypatch):
tests/test_app_home.py:24:def test_app_home_defaults_to_cwd():
tests/test_app_home.py:25:    assert ah.app_home() == Path(".")
tests/test_app_home.py:55:def test_app_home_honors_env(monkeypatch, tmp_path):
tests/test_app_home.py:57:    assert ah.app_home() == tmp_path
tests/test_app_home.py:60:def test_app_home_expands_user(monkeypatch):
tests/test_app_home.py:62:    assert ah.app_home() == Path.home() / "khelsutra-data"
tests/test_app_home.py:65:def test_app_home_ignores_blank_env(monkeypatch):
tests/test_app_home.py:67:    assert ah.app_home() == Path(".")
tests/test_app_home.py:93:# ---- default_os_app_home: the OS-conventional dir the launcher
exports (NOT auto-selected) ----

```

```

tests/test_app_home.py:95: def test_default_os_app_home_windows(monkeypatch, tmp_path):
tests/test_app_home.py:98:     assert ah.default_os_app_home() == tmp_path / "Roaming" /
"Khelsutra"
tests/test_app_home.py:101: def test_default_os_app_home_macos(monkeypatch):
tests/test_app_home.py:103:     assert ah.default_os_app_home() == Path.home() / "Library" /
"Application Support" / "Khelsutra"
tests/test_app_home.py:106: def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
tests/test_app_home.py:109:     assert ah.default_os_app_home() == tmp_path / "share" /
"Khelsutra"
tests/test_app_home.py:112: def test_default_os_app_home_linux_default(monkeypatch):
tests/test_app_home.py:115:     assert ah.default_os_app_home() == Path.home() / ".local" /
"share" / "Khelsutra"
tests/test_app_home.py:120: def test_loader_reads_config_from_app_home(monkeypatch, tmp_path):
tests/test_app_home.py:135: def test_save_default_config_writes_under_app_home(monkeypatch,
tmp_path):
tests/test_app_home.py:142: def test_explicit_path_still_overrides_app_home(monkeypatch,
tmp_path):
tests/test_app_launcher.py:136:     app_home = tmp_path / "apphome"
tests/test_app_launcher.py:137:     app_home.mkdir()
tests/test_app_launcher.py:141:         "RALLY_APP_HOME": str(app_home), # isolate: never touch
the real OS app-data dir
tests/test_app_launcher.py:149:     cfg = app_home / "config.json"
tests/test_doctor.py:3: doctor's --setup config write now follows
``backend.utils.app_home.default_config_path()`` ? the
tests/test_doctor.py:14: from backend.utils import app_home as ah
tests/test_doctor.py:26: def _clear_app_home_env(monkeypatch):
tests/test_doctor.py:30: def test_init_writes_config_under_app_home(monkeypatch, tmp_path):
tests/test_p2b_install.py:112: def test_default_os_app_home_per_os(install_mod):
tests/test_p2b_install.py:113:     win = install_mod.default_os_app_home(system="Windows",
env={"APPDATA": r"C:\Users\u\AppData\Roaming"})
tests/test_p2b_install.py:115:     lin = install_mod.default_os_app_home(system="Linux",
env={"XDG_DATA_HOME": "/home/u/.local/share"})
tests/test_p2b_install.py:122:         method="uv", app_home=home, shortcut=home /
"khelsutra.sh",
tests/test_p2b_install.py:152:         method="pip", app_home=home, shortcut=home /
"KhelSutra.bat",
tests/test_server_entrypoint.py:98: def test_app_home_roots_db_under_rally_app_home(tmp_path):
Binary file tests/__pycache__/test_app_home.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_app_launcher.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_doctor.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_p2b_install.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_server_entrypoint.cpython-313-pytest-9.0.3.pyc matches
=== parity / import backend in p2b test ===
9: from __future__ import annotations
11: import importlib.util
12: import json
13: from pathlib import Path
15: import pytest
17: from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
23:     spec = importlib.util.spec_from_file_location(name[:-3], path)
25:     mod = importlib.util.module_from_spec(spec)
70:     assert loaded.storage.backend == "local" # supplied by the truly-local preset

```

11. assistant (2026-06-22T01:31:25.377Z)

The no-parity-test claim is confirmed. Let me look at the relevant test lines to see exactly what each side tests, confirming they test the two mirrors independently with no cross-assertion.

12. user (2026-06-22T01:31:26.764Z)

```

108     pip = install_mod.install_command("pip", "badminton-highlight-indexer")
109     assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111

```

```

112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
r"C:\Users\u\AppData\Roaming"})
114         assert win.name == "Khelsutra" and "Roaming" in str(win)
115         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
"/home/u/.local/share"})
116         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
117
118
119     def test_build_manifest_shape(install_mod):
120         home = Path("/home/u/AppHome")
121         m = install_mod.build_manifest(
122             method="uv", app_home=home, shortcut=home / "khelsutra.sh",

```

13. user (2026-06-22T01:31:27.241Z)

```

93     # ---- default_os_app_home: the OS-conventional dir the launcher exports (NOT auto-
selected) ----
94
95     def test_default_os_app_home_windows(monkeypatch, tmp_path):
96         monkeypatch.setattr(ah.platform, "system", lambda: "Windows")
97         monkeypatch.setenv("APPDATA", str(tmp_path / "Roaming"))
98         assert ah.default_os_app_home() == tmp_path / "Roaming" / "Khelsutra"
99
100
101     def test_default_os_app_home_macos(monkeypatch):
102         monkeypatch.setattr(ah.platform, "system", lambda: "Darwin")
103         assert ah.default_os_app_home() == Path.home() / "Library" / "Application Support" /
"Khelsutra"
104
105
106     def test_default_os_app_home_linux_xdg(monkeypatch, tmp_path):
107         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
108         monkeypatch.setenv("XDG_DATA_HOME", str(tmp_path / "share"))
109         assert ah.default_os_app_home() == tmp_path / "share" / "Khelsutra"
110
111
112     def test_default_os_app_home_linux_default(monkeypatch):
113         monkeypatch.setattr(ah.platform, "system", lambda: "Linux")
114         monkeypatch.delenv("XDG_DATA_HOME", raising=False)
115         assert ah.default_os_app_home() == Path.home() / ".local" / "share" / "Khelsutra"
116

```

14. assistant (2026-06-22T01:31:38.175Z)

My verification is complete. The finding's facts fully hold up:

1. **Three independent mirrors exist and agree today.** Engine ``backend/utils/app_home.py::default_os_app_home`` (line ~109), ``scripts/install.py::default_os_app_home`` (line 47), and ``scripts/uninstall.py::default_os_app_home`` (line 35) all hardcode ``APP_DIR_NAME = "Khelsutra"``, use identical per-OS bases (``APPDATA`?Roaming / `Library/Application Support` / `XDG_DATA_HOME`?.local/share``), and append the same leaf. The script copies add injectable ``system`/`env`` params but their defaults reproduce the engine exactly. Verified byte-for-byte.
2. **The stdlib-only-bootstrap justification is sound.** Both scripts are documented as standalone bootstraps that must run before the package is installed (``install.py`` line 26: "stdlib-only so it runs as a standalone bootstrap BEFORE the package exists"), so they genuinely cannot import ``backend.utils.app_home``. The duplication is justified, not lazy.
3. **No parity test exists.** ``tests/test_p2b_install.py::test_default_os_app_home_per_os`` (line 112) tests only the script mirror; ``tests/test_app_home.py::test_default_os_app_home_*`` (lines 95-115) tests only the engine. Neither cross-asserts the two against each other. Confirmed via `grep` across ``tests/``.

****Assessment of severity:**** This is correctly classified as a nit, not a blocker or major. The risk is purely *future* drift ? there is zero functional defect today. The values are identical, both sides are independently tested, and the docstrings already cross-reference each other as deliberate mirrors. The suggested fix (a cross-import parity test) is a reasonable low-priority hardening, explicitly framed by the finding itself as "Optional / Low priority." It is a real, accurate observation about a latent maintenance gap, so ``is_real=true``, but it does not block and warrants only nit severity.

15. user (2026-06-22T01:31:44.071Z)

Structured output provided successfully

16. assistant (2026-06-22T01:31:48.075Z)

Verification complete. The finding holds up as a genuine but low-severity (nit) observation ? I confirmed the three mirrors agree today, the `stdlib-bootstrap` justification is sound, and no parity test guards against future drift. It is real but does not block.